

Introducción al lenguaje Common Lisp (ahora sí)

Héctor Galbis Sanchis

25 de enero de 2025

Índice

1. Introducción	1
2. Tipos de datos primitivos	2
2.1. Números	2
2.2. Booleanos	4
2.3. Caracteres y cadenas de texto	5
3. Control de flujo	6
4. Variables	8
5. Funciones	10
6. Hello world!	11
7. Conclusión	13

```
;; File setup  
(setq org-babel-default-header-args:lisp '(:results . "both"))
```

1. Introducción

Como ya sabrás, los lenguajes de programación nos sirven para comunicarnos con el ordenador y decirle cómo manipular datos. Es por ello que siempre vale la pena empezar viendo cómo funciona un lenguaje hablando sobre los tipos de datos que podemos manejar en él.

Tras esto lo común es ver la sintaxis del lenguaje para realmente decirle a ordenador cómo manipulamos estos datos, pero con Common Lisp eso no va a ocurrir porque los propios datos ya son la sintaxis del lenguaje.

Recordemos que para trabajar con Common Lisp se suele trabajar con un *REPL*. Este *REPL* es lo que obtenemos al ejecutar cualquier implementación de Common Lisp. En él introducimos expresiones y el *REPL* las evalúa y devuelve el resultado. Al principio da la sensación de que sólo sirve para probar expresiones sencillas, pero poco a poco uno va descubriendo su verdadero potencial.

Dicho esto, en esta introducción estaré usando continuamente el *REPL* para evaluar expresiones de Common Lisp, a no ser que se diga lo contrario.

2. Tipos de datos primitivos

Como cualquier otro lenguaje de programación, Common Lisp dispone de los tipos de datos más primitivos. Entre ellos podemos encontrar:

- Números
- Booleanos
- Caracteres y cadenas de texto

Hay muchísimos tipos de datos más que iremos introduciendo poco a poco.

2.1. Números

En cuanto a los números enteros no hay mucho misterio. Se escriben exactamente igual que en cualquier otro lenguaje:

```
5      ; El entero 5
-6     ; El entero negativo -6
```

Lo que quizás no es tan común es que estos números enteros pueden usar, si es necesario, *bignums*. Esto quiere decir que podemos escribir cualquier número sea lo grande que sea y todo seguirá yendo bien.

[illegible]

Aunque aún no hayamos introducido las funciones, la siguiente expresión ejecuta la función + (función de suma):

$$(+ \ 3 \ 4 \ 5)$$

12

Como se puede ver, para ejecutar una función debemos poner entre paréntesis la función (en este caso +) y seguidamente los argumentos (en este caso 3, 4 y 5). Podemos también multiplicar con la función *, dividir con /, y restar con -.

```
(* 9999 9999 99999 999988877 123456 6656)
```

8215414458948896050459819081728

Estas funciones aceptan cualquier tipo numérico, incluidos los números con coma flotante.

$$(/ \ 5.0 \ 2.0 \ 2.0) \quad ; \ 5.0 \ / \ 2.0 \ / \ 2.0$$

1.25

Como ya ocurre en otros lenguajes, podemos usar notación científica para denotar números en coma flotante:

```
(- 6.e10 5.6e6)
```

5.9994403e10

Un tipo de dato poco común y que debería estar presente en más lenguajes de programación son las fracciones:

$$(-8/5 \ 7/8 \ 1/3) \quad ; \quad 8/5 - 7/8 - 1/3$$

47/120

Common Lisp tiene el tipo `rational`. No lo confundas con la división que puedes encontrar en otros lenguajes como C++ o Java. Recuerda que para dividir dos números en Common Lisp usamos la expresión `(/ a b)` donde `a` y `b` son dos números. Por otro lado, la expresión `a/b` es la fracción `a` entre `b`. Mientras que la división acepta cualquier tipo de número (enteros, fracciones, números con coma flotante), las fracciones sólo pueden formarse a partir de dos enteros.

También podemos usar números complejos. Usamos `#c(5 1)`, por ejemplo, para denotar al número $5+i$. Utilizando la función de exponenciación `expt` y la variable `pi` podemos calcular el valor -1 con la famosa identidad de Euler: $e^{(\pi i)} = -1$.

```
(expt 2.718281828459045d0 (* pi #c(0 1)))
```

La `d` que hay al final del número en coma flotante sirve para indicar que queremos usar doble precisión (`double`).

Si quieres más información sobre los números en Common Lisp te recomiendo ir al libro *Common Lisp the Language*.

Por otro lado, si quieres ver qué operaciones matemáticas existen en Common Lisp puedes visitar el *HyperSpec*.

2.2. Booleanos

No existen unos valores que podamos identificar únicamente como booleanos, sino que Common Lisp hace algo parecido al lenguaje C. En C, el número `0` significa falso y cualquier otra cosa significa verdadero. Por su lado, en Common Lisp, `NIL` es usado para indicar falsedad y cualquier otra cosa significa verdadero. Aunque, en particular, se usa el valor `T` para referirse al valor booleano que indica verdad. Es decir los valores que entran dentro del tipo `boolean` estrictamente hablando son:

- `T`: Verdadero
- `NIL`: Falso

Como es de esperar tenemos disponibles las funciones que se corresponden con las puertas lógicas:

```
(not nil)
```

T

```
(and (or t nil) nil)
```

NIL

Quizás te estés preguntando por qué he escrito antes NIL y en el código he escrito nil. La respuesta es que da igual cuál uses, ambos son equivalentes, pues Common Lisp no distingue entre mayúsculas y minúsculas (*case-insensitive*). Normalmente el código se escribe en minúscula y los resultados se suelen escribir en mayúscula. Lo más seguro es que mezcle ambas formas de escribir los valores, pero debes de saber que al final son equivalentes y da igual cómo lo escribamos.

Si quieres saber más sobre estas operaciones lógicas puedes volver al libro Common Lisp the Language.

2.3. Caracteres y cadenas de texto

Los caracteres en Common Lisp se denotan por #\ seguidos del caracter que quieres representar. Por ejemplo, si queremos el caracter g utilizamos #\g.

```
#\g
```

Existen un conjunto de 96 caracteres presentes en el estándar de Common Lisp (es decir, cualquier implementación debe tenerlos). Eso quiere decir que existen caracteres que podrían o no estar dependiendo de la implementación que usemos, pero no nos vamos a meter ahí.

De entre esos 96 caracteres, 2 se refieren al espacio y a la nueva línea. Para referirnos a estos dos caracteres utilizamos lo que se llama un nombre de caracter. En particular, se usa #\Space y #\Newline (de nuevo, da igual usar minúsculas o mayúsculas). Puedes encontrar la lista completa de los caracteres en el HyperSpec. Sin embargo, los nombres de caracteres disponibles están en otra sección. Recuerda que los únicos nombres de caracter que están presentes en el estándar son los dos primeros (#\Space y #\Newline).

Tenemos bastantes funciones que trabajan sobre caracteres. Podemos comparar su orden, ver si dos caracteres son iguales, si es mayúscula o minúscula, si es un dígito, etc.

```
(char<= #\a #\h)
```

T

```
(char-upcase #\y)
```

```
#\Y
```

Puedes ver una lista de algunas de estas funciones en el HyperSpec.

Por otro lado tenemos las cadenas de caracteres. Como en otros lenguajes de programación, las cadenas de texto, o strings, son arrays de caracteres. Las cadenas de texto se representan mediante una sucesión de caracteres puestos entre un par de dobles comillas (").

```
"Hola"
```

Recordemos que habían dos caracteres 'especiales' con los que nos teníamos que referir con un nombre de caracter: #\Space y #\Newline. Al igual que no usarías nunca #\a para escribir una a en un string, tampoco vas a usar #\Space y #\Newline para insertar un espacio o una nueva línea en un string. Y tampoco podemos usar \b o \n. Lo que tenemos que hacer es insertar literalmente un espacio (usando la barra espaciadora) o una nueva línea (usando la tecla *Enter*).

```
(string-upcase "Pongo espacios entre la palabras.  
Y una nueva linea entre frases.")
```

```
"PONGO ESPACIOS ENTRE LA PALABRAS.  
Y UNA NUEVA LINEA ENTRE FRASES."
```

Al igual que con los caracteres, existen algunas funciones que manipulan los strings, y otras funciones que nos dan información. También podemos concatenar varios strings:

```
(concatenate 'string "Hola " "mundo")
```

```
"Hola mundo"
```

3. Control de flujo

La operación más primitiva de control de flujo en Common Lisp es `if`. Como en cualquier otro lenguaje de programación, recibe una condición, y dependiendo de si esa condición es verdadera o falsa ejecutará una expresión u otra.

```
(if condicion
  entonces
  en-otro-caso)
```

Algo importante a tener en cuenta es que cualquier expresión de Common Lisp debe devolver algo, e `if` no es una excepción. En particular, `if` devolverá el valor que devuelva la expresión `entonces` o `en-otro-caso` dependiendo del valor de `condicion`.

```
(if (> 5 0)
  "5 es positivo"
  "5 no es positivo")
```

```
"5 es positivo"
```

En este caso se ha evaluado la expresión `(> 5 0)` que devuelve `T`, y por tanto se devuelve la expresión `"5 es positivo"` que está situado en el lugar del `then`.

A partir de `if` se construye `cond`, que es la versión `if-else-if-else-if` de otros lenguajes.

```
(cond
  (cond1 expr1)
  (cond2 expr2)
  (cond3 expr3)
  ...)
```

Es decir, podríamos usar `if` para ver si un número es positivo, negativo o cero de esta forma:

```
(if (> -4 0)
  "-4 es positivo"
  (if (< -4 0)
    "-4 es negativo"
    "-4 es cero"))
```

```
"-4 es negativo"
```

Pero es más cómodo usar `cond`:

```
(cond
  (> 0 0) "0 es positivo")
  (< 0 0) "0 es negativo")
  (t      "0 es cero"))

"0 es cero"
```

En la expresión `cond` se van evaluando las condiciones, y en el momento que se encuentre algo que sea verdadero (es decir, que devuelva `T`) devuelve la expresión asociada a dicha condición. En este caso, se usa como tercera condición el valor `T`, de esta forma creamos un 'en-otro-caso' si el resto de condiciones falla.

Ahora es cuando debería hablar de bucles `while` y `for`, pero en Common Lisp no hay. Y antes de que te lleves las manos a la cabeza te diré que hay algo muchísimo mejor: la expresión `loop`. Sin exagerar, engloba todas las expresiones típicas de bucles como `while`, `for`, `do-while`, `repeat`, ... y añade incluso más potencia. Es un pelín compleja y se merece un post entero para ella sola. Aun así, si tienes curiosidad, puedes encontrar su funcionamiento explicado con bastante detalle en el libro *Common Lisp the Language*.

4. Variables

Las variables son una de las herramientas más básicas de la programación que nos permiten abstraer información. Dado un objeto relativamente complejo, podemos darle un nombre y así poder usarlo en cualquier otro momento.

Para crear variables globales podemos usar `defvar`, `defparameter` y `defconstant` (entre otros un poco más raros). La mayor diferencia entre estas 3 expresiones es qué ocurre cuando reevaluamos la expresión con la misma variable pero con un valor diferente. Pero por ahora, nos quedaremos sólo con `defvar`.

```
(defvar miVariable 10)

MIVARIABLE
```

Recuerda que cualquier expresión de Common Lisp debe devolver algo. En este caso se retorna un símbolo, algo que veremos en otro momento. Por ahora, se ha definido la variable `miVariable` y ya podemos usarla como queramos.


```
(+ miVariable 5)
```

```
15
```

Otro ejemplo:

```
(defvar otraVariable (+ miVariable 2))
```

```
(/ miVariable otraVariable) ; 10/12
```

```
5/6
```

En cuanto a las variables locales, hay que pensar que la localidad depende de algo. En lenguajes como C++ o Java las variables locales suelen serlo respecto de una función. Es decir, esa variable sólo puedes usarla dentro de la función. En Common Lisp es un poco diferente. Aquí, tenemos un poco más de control sobre cuándo se puede o no usar una variable. En particular, una variable (normalmente) será local con respecto a la expresión `let`.

```
(let ((saludo "Hola")
      (nombre "Juan"))
  (concatenate 'string saludo " ", " nombre "."))
```

```
"Hola, Juan."
```

Una vez finaliza `let`, las variables `saludo` y `nombre` dejan de existir (a no ser que todo el `let` esté dentro de otro `let` con esas mismas variables o existan variables globales con esos nombres). Si ya existían variables con esos nombres, lo que hace `let` es ensombrecerlas (*shadowing*). Por ejemplo, recuerda que antes hemos definido la variable `miVariable`.

```
miVariable
```

```
10
```

Usando `let` podemos ensombrecer la variable global:

```
(let ((miVariable "Ahora es un string"))
  (string-upcase miVariable))
```

```
"AHORA ES UN STRING"
```

Una vez finaliza la expresión `let` la variable global `miVariable` deja de estar ensombrecida y vuelve a ser disponible con su valor original:

```
miVariable
```

```
10
```

5. Funciones

Si con las variables podemos dar nombre a los objetos, con las funciones le damos nombre a una serie de expresiones. Ya hemos usado algunas funciones como `+`, `-`, `concatenate` o `string-upcase`. Pero también estamos interesados en crear nuestras propias funciones. Para ello utilizamos `defun`.

```
(defun suma2 (n)
  (+ n 2))
```

```
SUMA2
```

Al igual que pasaba con `defvar`, `defun` también devuelve el símbolo que representa a nuestra función. Cuando usamos `defun` debemos indicar el nombre de la función, que en este caso es `suma2`, unos argumentos de entrada como `n`, y luego el cuerpo de la función. En el cuerpo podemos escribir la cantidad de expresiones que queramos. Además, el valor devuelto por la función será el valor devuelto por la última expresión del cuerpo de la función.

```
(defun saludar (nombre)
  (let ((saludo (concatenate 'string "Hola, " nombre ".")))
    (princ saludo))
  nombre)
```

```
SALUDAR
```

En la función `saludar` recibimos un `nombre` con el que creamos un `saludo`. Imprimimos el saludo con la función `princ` (también existe la función `print`). Por último, escribimos `nombre` al final de la función para devolver el nombre que hemos recibido.

```
(saludar "Juan")
```

```
Hola, Juan.  
"Juan"
```

En los resultados, primero se muestra qué se imprime por pantalla y luego se muestra el valor devuelto por la función.

En otro momento veremos cómo sacarle el máximo provecho a las funciones, pues es posible indicar argumentos opcionales, e incluso una cantidad variable de argumentos.

6. Hello world!

Para terminar vamos a ver cómo podemos hacer un programa *Hello World*. Es claro que si en el *REPL* de *SLIME* escribimos simplemente "Hello world", el propio *REPL* nos mostrará lo mismo.

```
"Hello world"
```

```
"Hello world"
```

Pero eso no es lo que queremos. Un programa *Hello World* requiere al menos que escribamos el código en un fichero, que podamos cargar dicho fichero y así poder ejecutar una función que nos imprima "Hello World". Pues eso es exactamente lo que vamos a hacer ahora.

Los programas de Common Lisp se escriben en ficheros con extensión *.lisp*. Vamos entonces a crear un fichero en el directorio *HOME* (~) llamado *'hello-world.lisp'*. En él, escribiremos la definición de una función *main* que imprimirá "Hello world!".

```
;;; ~/hellow-world.lisp
```

```
(defun main ()  
  (princ "Hello world!"))
```

Ahora sólo falta cargar el fichero. Para ello usamos la función *load* desde nuestro *REPL* (sí, todo se hace desde el *REPL*).

```
(load "~/hello-world.lisp")
```

```
T
```

Si la función `load` devuelve el valor `T`, significa que todo ha ido bien. Eso, además, quiere decir que la función `main` ya se puede usar.

```
(main)
```

```
Hello world!  
"Hello world!"
```

Quizás te sorprendas de que se haya impreso dos veces `Hello world!`. Eso es porque la función `princ`, que también tiene que devolver algo, devuelve el objeto que ha recibido. En este caso, `princ` recibe el objeto `"Hello world!"`, por lo que primero se imprime `Hello world!`, y luego se muestra el valor devuelto por la función `main` que es el valor devuelto por `princ`, es decir, `"Hello world!"`.

Podríamos ir un paso más y generar un ejecutable, aunque es un pelín más complicado. Pero qué carajos, vamos a hacerlo.

Primero hay que tener en cuenta que en el estandar no se habla nada sobre ejecutables, por lo que su creación debe estar soportada por las implementaciones. Es más, cada implementación lo hará de una manera diferente. En el caso de SBCL, la creación de un ejecutable pasa por usar la función `sb-ext:save-lisp-and-die`. ¡Pero cuidado! Esta función debe ejecutarse cuando no hay hilos extra ejecutándose. Y, ¿cuál es el problema? SLIME ejecuta varios hilos al mismo tiempo. Es decir, no podemos usar esta función en Emacs con SLIME. Pero la solución es sencilla: No usar Emacs, sino usar la consola de comandos.

Abre la consola de comandos y ejecuta SBCL:

```
sbcl
```

Se te abrirá un *REPL* en la terminal. Primero, como hemos creado una sesión nueva de Common Lisp, nuestra función `main` no está disponible. Así que, de nuevo, cargamos el fichero.

```
(load "~/hello-world.lisp")
```

```
T
```

Por si acaso, comprueba que ya está disponible usándola:

```
(main)
```

```
Hello world!  
"Hello world!"
```

Si todo ha ido bien, ya podemos crear el ejecutable:

```
(sb-ext:save-lisp-and-die "hello-world.out"  
                          :toplevel #'main  
                          :executable t)
```

El primer argumento es el nombre del nuevo fichero que queremos generar. El resto de argumentos son conocidos como *keyword arguments*. Cada argumento empezando por ':', como :toplevel es un *keyword*. Y el valor que le sucede es el valor asociado a ese *keyword*, en este caso, #'main. Con el *keyword argument* :toplevel indicamos la función que queremos que se llame cuando ejecutemos el ejecutable '~/'hello-world.out'. Por último, con el *keyword argument* :executable indicamos si queremos que se cree un ejecutable, por eso le pasamos el valor t.

Cuando ejecutes esta función, la sesión de Common Lisp se termina. Es decir, se cierra el *REPL* y volverás a la entrada de comandos de la terminal. Sólo queda probar nuestro programa:

```
./hello-world.out
```

```
Hello world!
```

7. Conclusión

Con este post hemos echado un vistazo general a lo más básico de Common Lisp: Números, booleanos, variables, ... Pero no te relajes porque esto escala muy rápido.

:D